

LinTeX: Promoting Accessibility in LaTeX Documents

by
Mohammad Danyal

Progress Report



Department of
Computer and Information Sciences

November, 2025

Contents

1	Aims and Objectives	1
2	Summary of Related Work	2
2.1	Programming Language Comparisons	2
2.2	Library Comparison	3
2.3	Design Focus	4
3	Project Specification	5
4	Project Plan including summary of progress	7
4.1	Future Plans	7
4.2	End-to-End Prototype	7
4.3	User Study	8
5	Approach to Development	9
5.1	Methodology	9
5.2	Design	9
5.3	Implementation and Technology Ideas	10
5.4	Testing and Evaluation Plans	10

The page count of 12 is exclusive of bibliography and appendices.

1 Aims and Objectives

There is a lack of accessibility with LaTeX, largely due to usage of outdated PDFLaTeX as opposed to LuaLaTeX. Usage of this outdated TEX engine necessitates workarounds for desired features and accessibility in resulting PDF documents.

What can be done to make LaTeX documents more accessible?

LinTeX is a developing tool dedicated to combat this problem by focusing on assisting screen readers' abilities to read the PDF documents produced from LaTeX documents. It aims to cover this area through suggesting structured tagging throughout a document, suggesting components where it may be lacking. Additionally, it will request users to input appropriate alternative text ('alt text') for non-text components of the document, including figures, images, and tables. Further focus may be placed on features such as ensuring appropriate contrast in any colours in the document, appropriate Sans Serif fonts, and flexible document design, largely depending on what development time allows for.

Additionally, as a tool focused on promoting accessibility, it too should be accessible. Therefore, the tool will ensure strict adherence to the Web Content Accessibility Guidelines 2 (WCAG 2) [W3C, 2018] requirements with a focus placed on keeping it a simple and intuitive tool for users to make use of.

Why should I use LinTeX?

While the tool is not limited strictly to the field, it is aimed to assist the many academics that find difficulties in accessing reports and academic literatures. By utilising this tool, you can ensure a wider demographic of users can view your documents, without necessitating an especially large amount of research in accessibility; it has been done for you.

2 Summary of Related Work

2.1 Programming Language Comparisons

There were many considerations when deciding what programming language to write LinTeX in. Languages chosen were based on those that I had prior experience in, to ensure minimal hinderance from the language itself to the development of the project. An important factor is ensuring LinTeX's maintainability in the future. Therefore, only the languages which have actively maintained Parsing Expression Grammar (PEG) libraries will be considered, as can be seen in Table 1.1. The most recent update times provided in Table 1.1 are in reference to 24th November 2025.

Table 1.1: Languages and Related PEG Frameworks

Language	PEG Library Link	PEG Library is in Development
Python	https://pythonhosted.org/pyPEG2/	No, last updated in October 2015
Python	https://github.com/erikrose/parsimonious	Yes, last updated in November 2025
JavaScript	https://github.com/pegjs/pegjs	No, last updated in November 2019
JavaScript	https://peggyjs.org/	Yes, last updated in September 2025
Go	https://github.com/pointlander/peg	Yes, last updated in November 2025
Go	https://github.com/mna/pigeon	Yes, last updated in November 2025
C	N/A	N/A
Java	https://github.com/sirthias/parboiled	Yes, last updated in November 2025
C#	https://code.google.com/archive/p/peg-sharp/	No, last updated in May 2011
C#	https://github.com/otac0n/Pegasus	No, last updated in December 2023
C++	https://github.com/yhirose/cpp-peglib	Yes, last updated in July 2025
PHP	https://github.com/hafriedlander/php-peg	No, last updated in May 2014
TypeScript	https://github.com/metadevpro/ts-pegjs	No, last updated in November 2024
TypeScript	https://github.com/EoinDavey/tsPEG	Yes, last updated in November 2025
Dart	https://github.com/darmie/dart-peg	No, last updated in July 2015

Based on the current criteria, we can rule out C, C#, PHP, and Dart.

As many languages remain, there is a need for a stricter criteria: suitability to be utilised as a web application as well as compatibility with the University of Strathclyde's development web server (devweb) system.

Table 1.2: Languages and Web Development Support

Language	Web Development Support	DevWeb Support
Python	Yes	Yes
JavaScript	Yes	Yes
Go	Yes	No
C	No, except for rare exceptions	No
Java	Yes, but not generally used	Yes
C#	Yes, but not generally used	No
C++	Yes, but not generally used	No
PHP	Yes	Yes
TypeScript	Yes	Yes
Dart	Yes	Yes

It should be noted that while Go, C, C#, and C++ are not directly compatible with devweb, as referenced in Table 1.2, they can be supported through use of Common Gateway Interface (CGI) scripts, and consequently should not be fully disregarded for this alone. However, based on their support for web development, C, Java, C#, and C++ will have a lower priority for being a chosen language.

This leaves us with a focus placed on JavaScript, Go, and TypeScript. However, JavaScript will be removed from consideration as a strictly typed language is preferred for our context. Therefore, leaving Go and TypeScript as the main considerations for this project. Finally, as I have more experience and a stronger preference towards Go as opposed to TypeScript, it was the chosen programming language for the project.

2.2 Library Comparison

Go has two libraries that are used in relation to PEG systems: Pigeon by Martin Angers (mna), and peg by Andrew Snodgrass (pointlander). Direct comparisons with core components of both have been made in Table 1.3.

Table 1.3: Comparison Between PEG and Pigeon

	Pigeon	Peg
Installation	Requires running a <code>go install</code> command and setting up environment variables	Requires supporting bash scripting (through Windows Subsystem for Linux (WSL) for Windows users), as well as running a <code>go install</code> command, cloning the repository, and running <code>go generate</code> && <code>go build</code>
Documentation	Contains a Godoc page, wiki page, an examples subdirectory, and a detailed README.md providing what PEG features are supported, command line interface usage, installation, and examples	Contains a detailed README.md and a PEG file syntax file containing what PEG features are supported, installation steps, and example projects links
Generated Code	Generated Go code can be used to configure settings on the parser, view statistics or error information, debug, enable or disable memoization, and parse readers, files, and strings. Additionally, the generated code is well documented, with several comments surrounding usage of public functions	Generated Go code can be used to run the PEG script on a provided string, debug and output the Abstract Syntax Tree (AST), output errors in parsing, and enable or disable memoization
Examples	Provides three direct examples: detecting indentation in files, parsing JSON files, and a calculator example. Examples were kept short (in regards to PEG scripts) with the calculator and indentation examples showcasing usage via a main function	Provides two project links for examples on PEG usage: a natural date/time parser and a scientific names parser. Both projects have large PEG scripts, with most lines being simple to read. The natural date/time project was intuitive to use and understand program flow

While examples are relatively similar, the simplicity of installation for Pigeon as opposed to Peg, combined with the depths of documentation both in generated code and outwith it, makes Pigeon a more ideal option. Additionally, its greater number of features gives LinTeX more to make use of. Therefore, Pigeon was the chosen library for the project.

2.3 Design Focus

LinTeX will take strong inspiration from Integrated Development Environments (IDEs) with regards to its User Interface/User Experience (UI/UX) design. This is because LaTeX in structure, is similar to a programming language, even being Turing complete [Murrish, 2012]. Hence, an environment suitable to programming is suitable to LaTeX, with some special considerations put aside for its quirks. For example, depending on the user's screen size and preferences, they may wish to have the output of the LaTeX documents taking up half of the screen, a separate browser window or monitor, or out of the way until compilation.

3 Project Specification

Modern LaTeX documents utilise PDFLaTeX as opposed to LuaLaTeX. Thus, they often require workarounds to implement features and lack accessibility support. LinTeX is a new tool under development to combat this concern, which aims to improve LaTeX documents by avoiding workarounds and suggesting modern LaTeX usage as an alternative, with emphasis placed on accessibility that support screen readers.

Functional requirements include:

- LinTeX must allow one LaTeX file to be uploaded.
- LinTeX should allow multiple LaTeX files to be uploaded.
- LinTeX should allow non-LaTeX files (such as PDFs) to be uploaded.
- LinTeX must allow the files to be exported as a ZIP file once the user has finished using the system.
- LinTeX should allow the finished document to be previewed.
- LinTeX must detect if tagging exists or not.
- LinTeX must provide a fix by inserting tagging.
- LinTeX must detect when alt text is missing for images.
- LinTeX must detect when alt text is missing for figures.
- LinTeX must detect when alt text is missing for tables.
- LinTeX must provide a fix by inserting alt text for images.
- LinTeX must provide a fix by inserting alt text for figures.
- LinTeX must provide a fix by inserting alt text for tables.
- LinTeX should support batch error fixes for errors of the same type.
- LinTeX should reference further reading for any accessibility recommendations.
- The website should have all non-text content contain an alt text (WCAG 2.2: Guideline 1.1) [W3C, 2018].

- LinTeX should not be vulnerable to any of OWASP's 2024 top 10 vulnerabilities.
- LinTeX may be usable without a keyboard.
- LinTeX should provide simple error messages.
- LinTeX should provide simple suggestion messages.

Non-functional requirements include:

- The website should load within 2 seconds.
- The website should support a variety of standard desktop screen sizes.
- The website should support a variety of browsers.
- The website should be presentable in a simpler format if required (WCAG 2.2: Guideline 1.3) [W3C, 2018].
- The website should allow for content to be distinguishable including supporting resizing content and minimal colour usage (WCAG 2.2: Guideline 1.4) [W3C, 2018].
- The website may be operable through the keyboard alone (WCAG 2.2: Guideline 2.1) [W3C, 2018].
- The website should include navigation features (WCAG 2.2: Guideline 2.4) [W3C, 2018].
- The website should be predictable (WCAG 2.2: Guideline 3.2) [W3C, 2018].
- The website should follow the law of locality.
- LinTeX should provide a PDF document for several files in under a minute.
- LinTeX should create a ZIP of the documents in under a minute.
- LinTeX should have thorough documentation describing its usage and code.
- LinTeX should be able to handle several files with hundreds of lines each.
- LinTeX should be able to handle thousands of users using the system simultaneously.

4 Project Plan including summary of progress

4.1 Future Plans

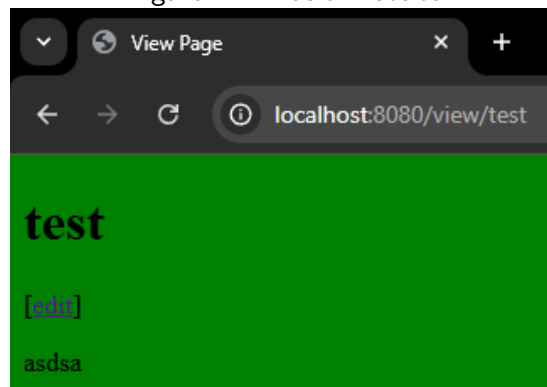
Focus will be shifted to completing the ethics application from 1st December 2025 to 14th December 2025. Once completed, it will be aimed to complete the user study during the second half of January, to give appropriate time to make any necessary adjustments to the ethics application.

When the ethics application is complete, focus will be shifted to completing the prototype and relevant research for it. Predictably, this would begin on 15th December 2025 or sooner. Ensuring basic functionality, with a focus on backend, will occur in the initial stages, likely requiring a week or two of dedicated efforts. Afterwards, Figma wireframes will be produced for the UI/UX to ensure it meets accessibility requirements which may take a week to complete. After suitable Figma diagrams have been created, implementation of the wireframes will take place which may take up to two weeks, due to a lack of experience with Tailwind CSS.

4.2 End-to-End Prototype

A basic webpage was created in Go utilising the Go package template (html/template), as showcased in Figure 4.1. This website was created following a tutorial on the Go webpage [Authors, 2010]. Additionally, it incorporated basic styling to confirm CSS could be ran.

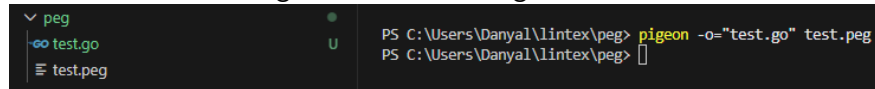
Figure 4.1: Basic Website



After confirming a basic webpage could be created, the next objective was confirming a PEG script could be ran to generate Go code. This was confirmed utilising a basic calculator

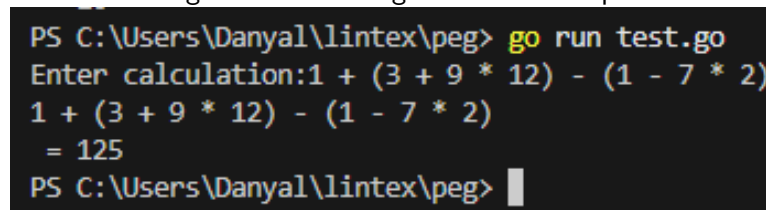
example PEG script provided in Pigeon documentation [Angers, 2025], which ran without issues as confirmed in Figure 4.2.

Figure 4.2: Generating Go Code



It was important to confirm the generated Go code was usable, which was confirmed by utilising a simple function to receive input from the user (the calculation to be performed), and outputted the correct amount by making use of test.go.

Figure 4.3: Running Basic PEG Script



Immediate next steps would be creating a basic PEG script which prints an output to console when specific LaTeX tags are detected. Additionally, it will be important to ensure it can be connected to the website to be ran. Therefore, a further step includes receiving user input from the website, running generated Go code from the PEG script on it, and outputting relevant information to the website. This would work as a minimum proof of concept, as it would confirm we can identify a missing LaTeX tag, for example missing alt text associated with an image.

4.3 User Study

A prototype Consent Form and Participant Information Sheet have been created, as can be viewed in the appendix. Next steps include finalising details for the ethics application as well as receiving further reviews.

5 Approach to Development

5.1 Methodology

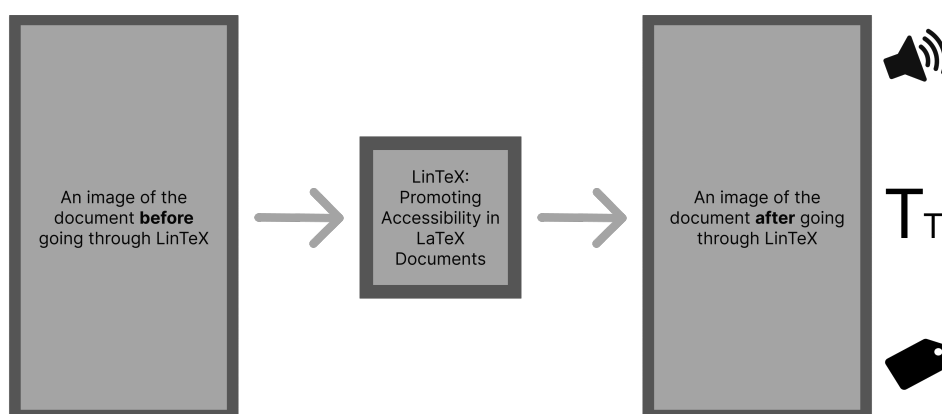
Test-Driven Development (TDD) is the chosen development methodology for this project. The reason for this is that there are a large number of features in LaTeX and involving testing at every stage ensures that each component can be handled in a methodological manner. Additionally, this is an opportunity to explore a development methodology that I have not had the opportunity to utilise beforehand, so it is an opportunity to learn.

TDD is a development methodology which often works in sprints similar to an Agile Development Methodology. However, it differs in that the test cases are written first and the code is then adapted to pass the test cases [Ozdil, 2021]. Therefore, it is imperative that the test cases written for LinTeX are thorough.

5.2 Design

The initial overview diagram, as showcased in Figure 5.1, was used to showcase a document lacking accessibility becoming more accessible after using LinTeX.

Figure 5.1: Overview Diagram Prototype



However, the prototype required the images within it to showcase the accessibility in a strong manner, which could be difficult to do with them alone. Therefore, the

overview diagram has become more focused on a panel-based design (i.e. a 'comic') utilising characters from the UK government's disability and impairment user profiles [Government Digital Service and Office, 2017]. There are several characters involved: Ashleigh, a partially sighted screenreader user; Christopher, a user with rheumatoid arthritis; and LinTeX, a printer-like character which takes LaTeX documents and makes them more accessible.

Ashleigh and Christopher will initially be having a conversation about difficulty with reading academic documents, with LinTeX easedropping on their conversation. LinTeX will then offer a solution by taking their document and putting it through his printer-like services to create a more accessible end document. Finally, Ashleigh and Christopher will comment on how it has made the document easier to read. For example, mentioning better accuracy with supporting screen readers.

5.3 Implementation and Technology Ideas

Go is the programming language of choice. A PEG library has been chosen as core to the project, with Pigeon being the chosen library to implement it [Angers, 2025]. Go package template (html/template) will be used to implement the webpages [Authors, 2025a], with Tailwind CSS used to create the styling. CGI scripts on devweb or an nginx web server will be used to run the code, with further considerations being necessary. Go package testing will be used to implement test cases [Authors, 2025b], with generated test cases ('fuzzing') used where possible.

5.4 Testing and Evaluation Plans

To evaluate the accessibility of the LaTeX document, and usability of LinTeX, a user study will be conducted. It will have several steps included: participants will be asked to undergo a think-aloud session where they will use LinTeX, as well as answer some questions about their experience after completing the think-aloud session. A net promoter score could be generated based on the questions asked.

References

- [Angers, 2025] Angers, M. (2025). pigeon - a peg parser generator for go. <https://pkg.go.dev/github.com/mna/pigeon>.
- [Authors, 2010] Authors, T. G. (2010). Writing web applications. <https://go.dev/doc/articles/wiki/>.
- [Authors, 2025a] Authors, T. G. (2025a). <https://pkg.go.dev/html/template>.
- [Authors, 2025b] Authors, T. G. (2025b). <https://pkg.go.dev/testing>.
- [Government Digital Service and Office, 2017] Government Digital Service, C. D. and Office, D. (2017). Understanding disabilities and impairments: user profiles. <https://www.gov.uk/government/publications/understanding-disabilities-and-impairments-user-profiles>.
- [Murrish, 2012] Murrish, R. (2012). Latex is more powerful than you think - computing the fibonacci numbers and turing completeness.
- [Ozdil, 2021] Ozdil, M. H. (2021). What is test-driven development? why tdd is so important?
- [W3C, 2018] W3C (2018). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>. A W3C Recommendation.

Participant Information Sheet for LinTeX User Study

[FOR USE WITH STANDARD PRIVACY NOTICE FOR RESEARCH PARTICIPANTS]

Name of department: Computer and Information Sciences

Title of the study: LinTeX: Promoting Accessibility in LaTeX Documents

Introduction

I am Mohammad Danyal, a final year Undergraduate Computer Science student at the University of Strathclyde. I can be contacted by email via mohammad.danyal.2023@uni.strath.ac.uk should any concerns or questions arise regarding the study, or if you wish to withdraw your participation.

What is the purpose of this research?

Modern LaTeX documents utilize PDFLaTeX as opposed to LuaLaTeX. Thus, they often require workarounds to implement features and lack accessibility support.

LinTeX is a new tool under development to combat this concern, which aims to improve LaTeX documents by avoiding workarounds and suggesting modern LaTeX usage as an alternative, with emphasis placed on accessibility.

This user study aims to identify the effectiveness of a new tool, LinTeX, to promote accessibility in LaTeX documents, as well as its usability to a wider audience. Additionally, this user study aims to identify areas of improvement for LinTeX.

Do you have to take part?

Participation in the research is entirely voluntary.

Refusing to participate in or withdrawing from the study will not affect the ways an individual is treated.

Participants may withdraw from the study at any stage without providing a reason.

If you wish to withdraw from the study at any stage, please contact mohammad.danyal.2023@uni.strath.ac.uk.

What will you do in the project?

Participants will be requested to participate in an in-person think-aloud user study in the University of Strathclyde's campus at a pre-arranged time and date.

Participants will be provided the LinTeX website to interact with and requested to create a simple LaTeX document with features selected to identify where common workarounds are necessary.

Participants may add more to the LaTeX document if they wish.

Participants will be observed interacting with the system with no interjection from the researcher unless necessary.

After completing the think-aloud session, participants will be asked about their experience including if they found the end document to be more accessible than the document before applying it to the system, as well as any adjustments they would have found useful.

Why have you been invited to take part?

Participants of the research study may benefit from the LinTeX tool and have been asked to assist in its development through an in-person think-aloud user study.

Participants must be a member of the University of Strathclyde, or another higher education facility within travelable distance.

Participants will require basic knowledge of LaTeX to participate in the study and have been selected based on these criteria.

What information is being collected in the project?

Personal information that may be stored include the participants' names and email addresses. This data is stored to ensure communication is thorough in arranging a time and date to participate in the study. Additionally, some information stored about the participants' usage of the system or answers to questions may be personal.

Non-identifiable information that may be stored include anonymised information extracted from the user study.

Who will have access to the information?

Only the researcher will have access to any Personally Identifiable Information (PII). Anonymised information may be included in the research paper and shared with the supervisor.

Where will the information be stored and how long will it be kept for?

All data will be stored in password-protected folders on OneDrive, with Personally Identifiable Information (PII) and anonymous data stored in separate folders.

All data stored in these folders will be deleted on 1st August 2026. However, any data that is anonymised and included in the study may not be possible to remove.

Thank you for reading this information – please ask any questions if you are unsure about what is written here.

All personal data will be processed in accordance with data protection legislation. Please read our [Privacy Notice for Research Participants](#) for more information about your rights under the legislation.

What happens next?

If you are interested in the project, or continuing with the user study, please contact me by email via mohammad.danyal.2023@uni.strath.ac.uk. Participants will be requested to sign a consent form prior to participation in the user study.

All information collected in the user study will be anonymised before being published.

If participants are interested in participating in any future related studies, the aforementioned email address can be used to register your interest.

If you are no longer interested in participating in the project, no further steps will be required. Thank you for your time.

Researcher contact details:

Mohammad Danyal

mohammad.danyal.2023@uni.strath.ac.uk

Chief Investigator details:

Brian Mitchell

brian.mitchell@strath.ac.uk

This research was granted ethical approval by the Department of Computer and Information Sciences Ethics Committee.

If you have any questions/concerns, during or after the research, or wish to contact an independent person to whom any questions may be directed or further information may be sought from, please contact: cis-ethics@strath.ac.uk

Consent Form for LinTeX User Study

Name of department: Computer and Information Sciences

Title of the study: LinTeX: Promoting Accessibility in LaTeX Documents

Please check each item to indicate agreement.

- ☐ I confirm that I have read and understood the Participant Information Sheet for the above project and the researcher has answered any queries to my satisfaction.
- ☐ I confirm that I have read and understood the [Privacy Notice for Participants in Research Projects](#) and understand how my personal information will be used and what will happen to it (i.e. how it will be stored and for how long).
- ☐ I understand that my participation is voluntary and that I am free to withdraw from the project at any time, up to the point of completion, without having to give a reason and without any consequences.
- ☐ I understand that I can request the withdrawal from the study of some personal information and that whenever possible researchers will comply with my request. This includes the following personal data:
 - ☐ My personal information collected from the think-aloud session.
 - ☐ My personal information from answers to questions.
 - ☐ My personal information from transcripts.
- ☐ I understand that anonymised data (i.e. data that do not identify me personally) cannot be withdrawn once they have been included in the study.
- ☐ I understand that any information recorded in the research will remain confidential and no information that identifies me will be made publicly available.
- ☐ I consent to being a participant in the project.

(PRINT NAME)	
Signature of Participant:	Date: